

Interaktiva visualiseringar

Syfte

Syftet med det här projektet var att fördjupa mig inom Plotly: att läsa på och se hur biblioteket kan användas för att göra olika sorters interaktiva visualiseringar och hur man kontrollerar dem, att kunna para ihop och skapa rätt interaktivitet för rätt funktion. Jag arbetade medvetet inte med animationer eller 3D-grafer för att avgränsa projektet till att fokusera på interaktivitet. Tester med pytest är till för att kontrollera att figurerna har rätt datastruktur, alltså har de inget med formen eller utseendet av figurerna att göra.

Området och dess relevans

Jag valde att fördjupa mig inom ett datavisualiseringsbibliotek för Python, Plotly. Det är ett open source-bibliotek för att skapa interaktiva och läsbara figurer. Data science-fältet handlar inte bara om att skriva kod, utan även att kunna förmedla resultat till andra människor. Här kan Plotly vara till stor nytta, då den lätta interaktiviteten i biblioteket gör att mottagaren, till exempel en uppdragsgivare eller kund, själv kan undersöka datan. Genom att inte bara ha ett statiskt diagram utan ett interaktivt, kan läsaren ställa och svara på sina egna frågor genom att hovra, zooma eller markera i figuren.

Viktiga begrepp

Biblioteket består av två delar där JavaScript-biblioteket [plotly.js](#) ritar upp figuren i webbläsaren efter att ett lager kod i Python har beskrivit den. Visualiseringen är uppbyggd av innehåll med data och en layout, som beskriver inställningarna man valt, exempelvis titel och axlar.

Ett av de mest centrala begreppen i rapporten är `fig.data`, en lista med spår (traces) och varje spår där motsvarar en dataserie. I mitt projekt har stapel- och linjediagrammen ett spår medan spridningsdiagrammet har fyra, ett per kategori.

Två metoder som används i projektet som har påverkan på hur interaktiviteten för varje diagram ser ut är `update_traces` och `update_layout`. `Update_traces` anropas i alla tre funktionerna, eftersom `hovertemplate` hör till spåret och därmed beskriver en enskild

dataserie. I stapeldiagrammet används `update_layout` för separators och i linjediagrammet för hovermode. Med `x unified` räcker det att man är nära rätt `x`-värde, så visas en ruta för det datumet med datum och värde. Till skillnad från `hovertemplate` kan hovermode inte skilja på serier i samma diagram och hör därför till layouten.

En central import som görs i projektet är `import plotly.express`, som anropas i de tre diagrammens funktioner. Denna modul bygger visualiseringen åt användaren, inklusive en `hovertemplate`, som i stapeldiagrammets fall var:

```
product_category=%{x}<br>order_value=%{y}.
```

Genomförande - tempus dåtid

För att genomföra arbetet använde jag främst Plotly-biblioteket, tillsammans med `pandas`, `pytest` och `kaleido`. `Kaleido` behövs endast för att exportera PNG-filer i projektet.

Datan som användes i projektet är en `orders.csv`-fil som jag fått i en parallell kursuppgift. Den passade bra eftersom jag redan var insatt i den och kände till kolumnerna väl, dessutom innehåller den 80 ordrar med både kategorier, datum, pris och antal, vilket gjorde det möjligt att bygga tre olika sorters diagram.

Projektet inleddes med att skapa ett minimalt bevis (`proof of concept`) för att kontrollera att grundläggande funktioner fungerade, nämligen att biblioteket installerades, att en figur skapades, att PNG-exporten gav en bild och att det gick att genomföra ett test med `fig.data`. Den sista punkten var avgörande, eftersom hela testet byggde på att figurerna gick att läsa som objekt.

Lösningen fungerar genom att [main.py](#) läser och förbereder datan samt anropar figurfunktionerna och sparar ner dem i HTML- och PNG-format i output-mappen. Filen [prepare.py](#) ansvarar för datatvätt, där den normaliserar namn, gör `order_date` till datum samt lägger till `order_value`. Denna fil är skild från [charts.py](#) som istället står för figurbyggande. Det gör att funktionerna kan testas mot känd, kontrollerad data.

Kontrollen av indata låg först inskriven i varje figurfunktion. När samma rader stod på tre ställen bröt jag ut dem till en egen funktion, `_validate_orders`, som de tre anropar. För att göra det möjligt att testa figurfunktionerna kodade jag om dem så att de inte sparar ner någonting utan endast returnerar figurer, då de kan kontrolleras direkt i minnet utan att filsystemet behövs. Tester med `pytest` skrevs för att verifiera att figurerna är strukturerade korrekt och innehåller rätt data. Testerna kontrollerar: innehåll, saknade kolumner, tom data, `hovertemplate` och spår per kategori.

Dessutom valde jag att skapa tre diagramtyper som beter sig på olika sätt: i stapeldiagrammet handlar interaktionen om avläsning, i linjediagrammet om navigering och i spridningsdiagrammet om urval.

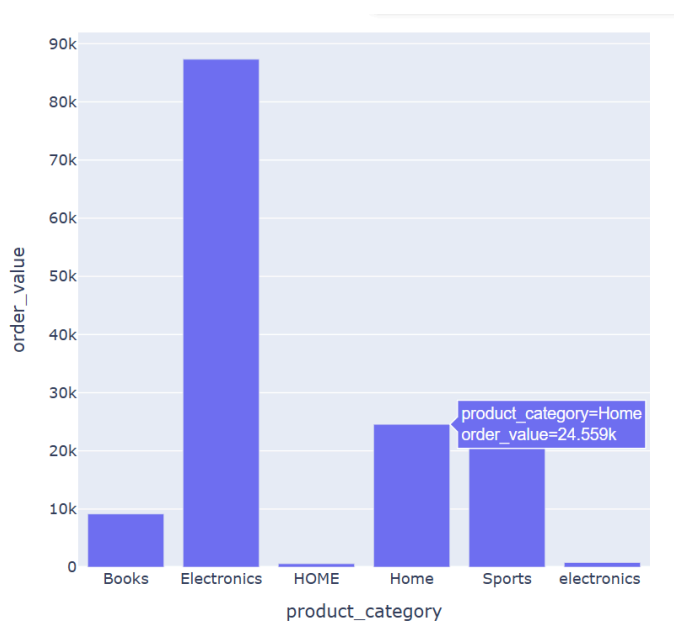
Jag stötte på ett antal problem under projektets gång. När jag skapade diagrammet summerades `unit_price`. Vilket blev fel mått då kolumnen inte räknade summan av försäljning utan enskilda prislappar. För att lösa problemet la jag in `order_value` i [prepare.py](#) för att beräkna `quantity * unit_price`, så att diagrammen faktiskt summerar vad orderarna var värda.

Tidigt i projektet när jag skapade mitt första stapeldiagram insåg jag att olika stavning av kategorinamnen i csv-filen gav olika staplar, exempelvis HOME och Home. Lösningen på detta var även här att lägga in normalisering i `prepare.py` för att göra skrivsätten enhetliga, så att varje kategori blev en stapel.

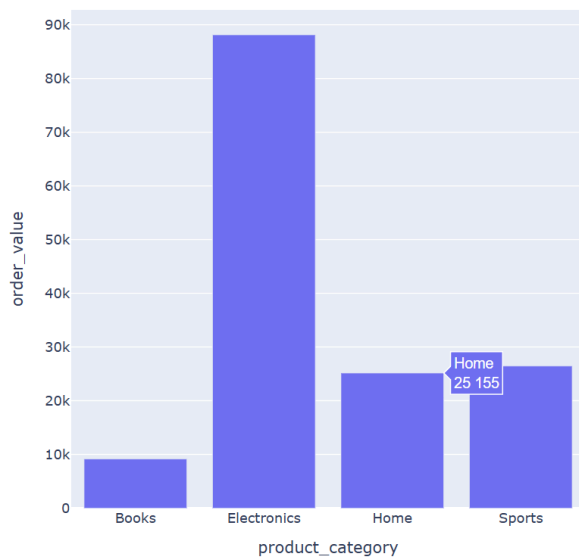
När jag skapade spridningsdiagrammet visade tooltipen bara två tal utan kontext, ingen etikett som visade vad talen betydde och ingen kategori. Jag hade kopierat in `hovertemplate` från stapeldiagrammet, där jag inte behövde mer information än siffror och det som står på axeln. Lösningen blev två ändringar. Jag skrev in etiketter i mallen och utelämnade `<extra></extra>`, som tar bort rutan med spårets namn. Det gjorde att rutan blev klar och i det här diagrammet visar den kategorinamnet eftersom det har fyra spår.

Resultat

Stapeldiagram



Figur 1. Stapeldiagrammets tooltip med Plotly Express automatiskt mall. Kolumnnamnen product_category och order_value skrivs ut och ordervärdet som visas är 25.155k.



Figur 2. Samma diagram men med egen hovertemplate. Kolumnnamnen är nu borta och beloppets värde visas i sin helhet.

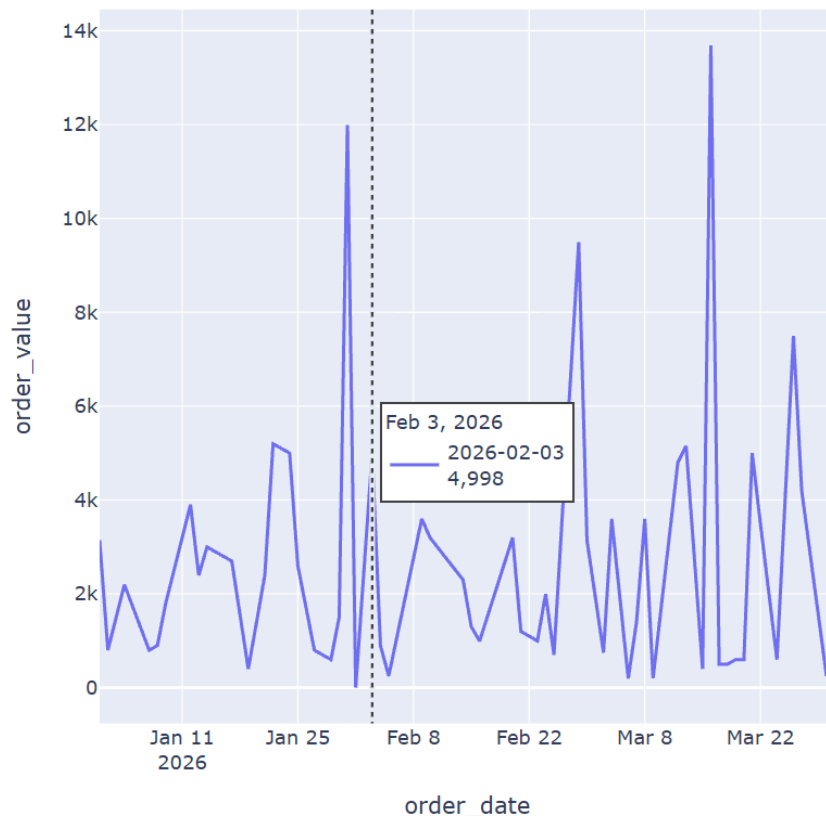
```
fig.update_traces(hovertemplate="%{x}<br>{%y:,.0f}<extra></extra>")
```

Den här raden ersatte Plotly Express automatiska mall. `%{x}` ger kategorin, `%{y:,.0f}` ger beloppet med avgränsare och `<extra></extra>` tar bort rutan med spårets namn, som är tom i diagrammet då det bara är ett spår.

```
fig.update_layout(separators=", ")
```

Här används `update_layout` för att ordervärdet ska skrivas med ett mellanslag, för att göra det tydligare.

Linjediagram



Figur 3. Linjediagrammet visar försäljningen över tid. Vid hovring visas en ruta med datum och summerat orderbelopp för det datumet.

```
fig.update_traces(hovertemplate="%{x|%Y-%m-%d}<br>{%y:,.0f}<extra></extra>")  
fig.update_layout(hovermode="x unified")
```

Mallen är densamma som stapeldiagrammet, med tillagt datumformat `%{x|%Y-%m-%d}`. Utan specifikation för formatet väljer [plotly.js](https://plotly.com/javascript/) själv hur datumen presenteras.

Hovermode lades till tillsammans med `update_layout`, eftersom den hör till diagrammet som helhet och inte till en enskild dataserie. Med `x unified` räcker det att man är nära rätt x-värde, så visas en ruta för datumet. Det blir användbart när man har flera datum, så man slipper pricka exakt rätt punkt.

Spridningsdiagram



Figur 4. Spridningsdiagrammet visar enskilda ordrar. Varje punkt är en order, som är placerad efter styckpris på x-axeln och enheter på y-axeln. Eftersom quantity bara antar värdena 1-4 hamnar punkterna på endast fyra olika nivåer i rad.

.Färgerna anger produktkategori och kommer av:

```
color="product_category"
```

i px.scatter.

```
fig.update_traces(hovertemplate="Price: %{x}<br>Quantity: %{y}")
```

En punkts läge säger inte vilket tal som är pris och vilket som är antal, därför skrev jag in etiketter. Rutan med kategorinamnet behövs vid fler än ett spår, och därför utelämnades `<extra></extra>`. Hovrar man över en punkt ser man styckpriset och hur många varor kunden köpte i den ordern, till exempel tre elektronikvaror för 1299 styck.

Begränsningar och möjliga förbättringar

Pytesterna läser figurobjektet i Python, men ritandet sker i [plotly.js](https://plotly.com/python/) webbläsare, vilket inte går att nå från Python. Jag kan verifiera att hovertemplate finns men inte om tooltipen visar rätt.

Interaktiviteten i projektet sker inom ett färdigt diagram, där användaren kan utforska det som visas. Däremot kan inte användaren välja vad som visas med filter och kopplade

diagram. Det krävs i så fall en server. Plotlys dokumentation hänvisar vidare till Dash, som bygger på samma bibliotek men som gör det möjligt för användaren att styra det som visas. Att lägga till Dash är en möjlig utveckling.

Ett spridningsdiagram fungerar bäst när båda axlarna kan anta många olika värden. Kolumnen quantity kan bara vara 1-4, så punkterna hamnar endast på fyra rader istället för att sprida ut sig. Datan valdes för att jag kände väl till den och kunde lägga tiden på Plotly, men med data där båda värdena varierar mer hade diagramtypen passat bättre. Här visar det framför allt hur color delar upp punkterna i en serie per kategori. En annan möjlig förbättring vore att visa orderns totalbelopp i spridningsdiagrammets tooltip. Har två ordrar exakt samma pris och antal hamnar de på samma punkt, vilket gör att diagrammet ser ut att innehålla färre ordrar än det faktiskt gör. I efterhand har jag tänkt att det skulle kunna lösas genom att göra punkterna halvgenomskinliga, så att överlappande punkter blir mörkare, eller genom att låta punktens storlek visa hur många ordrar som ligger där.

Koppling till yrkesrollen

Att sätta sig in i nya bibliotek är något man gör om och om igen i yrket. I det här fallet gällde det Plotly, men arbetssättet är detsamma: fråga, bevis och kontroll.

I det här projektet byggde jag en återanvändbar modul som returnerar färdiga figurer. En analytiker som gör samma sorts rapport varje månad kan bygga diagrammet en gång och sedan anropa det, istället för att skriva om det varje gång.

Till skillnad från kod som kraschar ger en felaktig figur inget felmeddelande. Den ritas upp som vanligt och användaren litar på den genererade bilden. Vid återkommande rapportering är det bra att kontrollera att figuren innehåller det som den ska.

Källor

Plotly (u.å.a) Hover text and formatting in Python. [Hover text and formatting in Python](#) (21-9-2026)

Plotly (u.å.b) Plotly - Github. [Plotly](#) (20-9-2026)

Självreflektion

1. Vad lärde du dig som du inte kunde innan?

Jag lärde mig hur man skapar tre olika sorters interaktiva figurer med Plotlys bibliotek för Python. Även hur man kontrollerar figurernas innehåll och att de består av rätt delar med hjälp av tester med pytest.

2. Vad var svårast att förstå eller genomföra?

Jag tyckte det var svårast att förstå koden för hovertemplate, varför den skulle se ut på ett visst sätt, och hur jag skulle anpassa innehållet för att passa till varje figur.

3. Vilket tekniskt val är du mest nöjd med och varför?

Jag är mest nöjd med att jag strukturerade uppgiften och koden på ett bra sätt, där jag först skapade ett stapeldiagram för att se vad Plotly Express kunde generera och sedan anpassade jag det för mitt syfte och efter mina behov som sedan kontrollerades med pytest.

4. Vad hade du gjort annorlunda om du började om?

Jag hade valt data med variabler med fler möjliga värden, så att spridningsdiagrammet inte blir som på fyra linjer, då i mitt fall quantity bara kunde vara 1-4.

5. Vad skulle vara ett naturligt nästa steg om du fortsatte arbetet?

Fortsätta att testa Plotly på annan data, vilket öppnar upp möjligheter för att skapa andra figurer med mer och annan slags interaktivitet. En annan möjlighet är att lägga till Dash, som Plotlys dokumentation hänvisar till, som gör det möjligt för användarna att styra vad som visas i visualiseringarna.

6. Vilket betyg tycker du själv att arbetet motsvarar – G eller VG?

G

7. **Motivera din bedömning genom att koppla till kraven för G och VG nedan.**

Jag har skrivit en fungerande kod och lärt mig ett nytt bibliotek med hjälp av källor relaterade till Plotly. För att nå VG hade jag behövt förklara mer ingående hur Plotly fungerar och inte bara hur det används.